

NGINX - Troubleshooting y Guía de Uso

¿Es NGINX obligatorio?

No. NGINX es **opcional** y solo se usa como balanceador de carga entre las dos instancias de la API directa (:8000 y :8001). Si NGINX no está disponible o falla, el sistema sigue funcionando — simplemente accedes directamente a los puertos de cada API.

Modo sin NGINX (desarrollo local)

Si no quieres usar NGINX, puedes:

Opción 1: Usar `start_without_nginx.sh`

```
./scripts/start_without_nginx.sh
```

Levanta todo el sistema sin intentar arrancar NGINX.

Opción 2: Usar `start_all.sh` (NGINX es opcional)

```
./scripts/start_all.sh
```

Si NGINX no está instalado o falla, el script continúa sin él y te avisa.

Acceder a las APIs directamente

```
# API Directa - instancia 0
curl http://localhost:8000/health

# API Directa - instancia 1
curl http://localhost:8001/health

# Gateway Indirecto
curl http://localhost:8080/health
```

Ejecutar benchmarks sin NGINX

```
# Apunta directamente al puerto de la API
./scripts/run_direct_benchmark.sh --base-url http://localhost:8000
```

Modo con NGINX (balanceo de carga)

Requisitos

```
# Instalar NGINX (Ubuntu/Debian)
sudo apt install nginx

# Verificar instalación
nginx -v
```

Cómo funciona

NGINX escucha en el **puerto 80** y distribuye las peticiones entre:

- 127.0.0.1:8000 (API Directa #0)
- 127.0.0.1:8001 (API Directa #1)

Usa la estrategia **least_conn** (envía al servidor con menos conexiones activas).

Ejecutar con NGINX

```
./scripts/start_all.sh
# 0
./scripts/start_direct_only.sh
```

Verificar que NGINX está activo

```
curl http://localhost:80/health
# o simplemente:
curl http://localhost/health
```

Problemas comunes

1. `open() ".../mime.types" failed (2: No such file or directory)`

Causa: Faltaba el archivo `mime.types` en `deploy/nginx/`.

Solución: El archivo ya está incluido en `deploy/nginx/mime.types`.

Si por alguna razón falta, puedes copiarlo del sistema:

```
# Copiar desde la instalación de NGINX del sistema
cp /etc/nginx/mime.types deploy/nginx/mime.types
```

2. `nginx: [emerg] bind() to 0.0.0.0:80 failed (13: Permission denied)`

Causa: El puerto 80 requiere permisos de superusuario.

Solución:

```
# El script ya usa sudo, pero si lo ejecutas manualmente:
sudo nginx -c /ruta/completa/deploy/nginx/nginx.conf -p /ruta/completa/deploy/nginx/
```

3.

nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)

Causa: Otro proceso (o una instancia previa de NGINX) ya usa el puerto 80.

Solución:

```
# Ver qué usa el puerto 80
sudo lsof -i :80

# Parar NGINX si ya corre
sudo nginx -s stop

# O matar el proceso
sudo fuser -k 80/tcp
```

4. NGINX arranca pero las APIs no responden a través del puerto 80

Causa: Las APIs no están corriendo en los puertos 8000/8001.

Solución:

```
# Verificar que las APIs están corriendo
curl http://localhost:8000/health
curl http://localhost:8001/health

# Si no responden, revisar los logs
cat logs/direct_api_0.log
cat logs/direct_api_1.log
```

5. nginx: [emerg] "upstream" directive is not allowed here

Causa: El archivo `upstream.conf` tiene un error de sintaxis o ruta incorrecta.

Solución: Verificar que `upstream.conf` existe en `deploy/nginx/` y tiene el formato correcto:

```
upstream direct_backend {
    least_conn;
    server 127.0.0.1:8000 max_fails=3 fail_timeout=10s;
    server 127.0.0.1:8001 max_fails=3 fail_timeout=10s;
}
```

Diferencias: Con NGINX vs Sin NGINX

Aspecto	Con NGINX	Sin NGINX
Punto de entrada	<code>http://localhost:80</code>	<code>http://localhost:8000</code>
Balanceo de carga	✓ least_conn automático	✗ Manual (elige puerto)
Tolerancia a fallos	✓ Si una API cae, usa la otra	✗ Debes cambiar de puerto
Benchmarks base_url	<code>http://localhost</code>	<code>http://localhost:8000</code>
Requisitos	NGINX instalado + sudo	Ninguno extra
Recomendado para	Producción, benchmarks	Desarrollo, debug

Archivos de configuración

deploy/nginx/

 nginx.conf

 upstream.conf

 mime.types

 Configuración principal de NGINX

 Definición de backends (puertos 8000, 8001)

 Tipos MIME para respuestas HTTP